

JUSTIFICACIÓN DE LAS DECISIONES DE DISEÑO EN UN SISTEMA DE GESTIÓN

UNIVERSIDAD EL BOSQUE

Bases De Datos I

Integrantes:

Gabriel Santiago Amortegui Hernández

David Santiago Gomez Duran

Mariana Camila Durán Piñeros

Profesor:

Christian Felipe Duarte Arevalo

Criterios utilizados para el diseño del sistema.

El diseño del sistema se realizó teniendo en cuenta los requerimientos establecidos para el proyecto y buscando que el programa fuera claro y funcional. Por este motivo, se definió un modelo sencillo, en el que se incluyeron únicamente los atributos necesarios para que el sistema pudiera operar correctamente.

Al momento de seleccionar la información a almacenar, se consideraron los siguientes criterios:

- **Cumplimiento de los requisitos del proyecto:** los atributos permiten implementar las funciones solicitadas, como el manejo de la información principal y los procesos básicos del sistema.
- **Simplicidad del modelo:** se evitó agregar información que no tuviera un uso directo, con el fin de mantener un diseño fácil de entender y manejar.
- **Relevancia de los datos:** cada atributo fue elegido porque aporta información necesaria para identificar los elementos del sistema y relacionarlos entre sí.
- **Cobertura de casos comunes:** aunque el modelo no es complejo, permite manejar las situaciones habituales que se presentan durante el uso del sistema, manteniéndose dentro del alcance del trabajo.

Modelo de datos y justificación de los atributos.

Usuario:

La clase representa a las personas que interactúan con el sistema y realizan préstamos de libros. Su diseño se enfocó en permitir una identificación clara del usuario y en facilitar su relación con los préstamos, sin convertirla en un registro detallado de información personal.

El atributo **id** se definió como identificador único del usuario dentro del sistema. Este atributo es fundamental para garantizar que cada usuario pueda ser referenciado de manera precisa, especialmente al momento de asociarlo con los préstamos realizados. Además, el uso de un identificador numérico simplifica el manejo de la información y la persistencia en archivos planos.

El **nombre** se incluyó para permitir una identificación legible del usuario. Este dato es necesario para mostrar información comprensible en búsquedas, listados y consultas, facilitando la interacción con el sistema sin depender únicamente de identificadores numéricos.

El atributo **correo** se agregó como un dato adicional de identificación. Aunque el sistema no maneja autenticación ni notificaciones, el correo permite diferenciar usuarios en casos donde

puedan existir coincidencias en los nombres y representa un dato común y razonable en este tipo de aplicaciones.

No se incluyeron otros atributos como dirección, teléfono u otros datos personales, ya que el sistema no requiere esa información para cumplir sus funciones. El objetivo de la clase Usuario es servir como referencia dentro de los procesos de préstamo, no almacenar información personal extensa. De esta manera, el modelo se mantiene enfocado, claro y acorde al alcance del proyecto.

Libro:

La clase Libro representa los libros que el sistema gestiona para su préstamo. Al definir sus atributos, se buscó que el sistema pudiera identificar cada libro de forma clara y controlar su disponibilidad, sin ampliar el modelo a una descripción bibliográfica completa.

El atributo **id** se incluyó como identificador interno del libro. Este atributo permite manejar los libros de manera ordenada dentro del sistema y facilita su asociación con los préstamos, asegurando que cada operación se realice sobre el libro correcto.

El **título** se definió como el principal dato descriptivo del libro. Este atributo resulta esencial para las búsquedas y para la visualización de la información, ya que es la forma más directa en la que los usuarios reconocen un libro.

El **autor** complementa la información del título y permite distinguir libros que pueden tener nombres similares. Su inclusión mejora la claridad de los resultados y aporta contexto al momento de consultar la información.

El atributo **isbn** se incorporó como un identificador estándar del libro. Su uso permite evitar registros duplicados y aporta mayor precisión en la identificación, especialmente cuando otros datos descriptivos pueden coincidir.

El atributo **disponible** cumple una función operativa dentro del sistema, ya que indica si un libro puede ser prestado en un momento determinado. Esta información es necesaria para validar los préstamos y mantener la coherencia en el uso de los libros.

No se incluyeron otros atributos como género, editorial o año de publicación, ya que el sistema está enfocado en el control del préstamo y la disponibilidad de los libros, y no en la administración detallada de un catálogo. De esta forma, el modelo se mantiene alineado con el propósito del sistema y con el alcance del proyecto.

Préstamo:

Representa la operación principal del sistema, ya que es el punto donde se relacionan los usuarios con los libros. Su diseño se enfocó en controlar esta relación de manera clara y ordenada, sin agregar reglas adicionales que no hacen parte del alcance del proyecto.

El atributo **id** permite identificar cada préstamo de forma única dentro del sistema. Esto facilita su gestión y evita conflictos cuando un usuario realiza múltiples préstamos a lo largo del tiempo.

El **idUsuario** se incluyó para establecer la relación entre el préstamo y el usuario que lo realiza. Esta decisión permite mantener una referencia directa sin depender de información adicional, lo cual simplifica tanto el manejo del sistema como la persistencia en archivos planos.

El **idLibro** permite que un préstamo se vincule con el libro que se va a prestar. De esta forma, se modela una situación común de manera más eficiente.

El atributo **fecha préstamo** se incluyó para registrar el momento exacto en el que se realiza la operación. Este dato permite llevar un seguimiento básico de los préstamos realizados, facilita la organización cronológica de los registros y aporta información útil para consultas posteriores, como el historial de uso del sistema.

El atributo de **fecha de devolución** también nos ayuda a gestionar si el préstamo sigue activo o no, ya que si no tiene fecha de devolución definida significa que el préstamo sigue activo de lo contrario se entiende que no sigue activo.

Tomamos la decisión de no agregar ni manejar el atributo de estado para simplificar las acciones y poder validar ese mismo atributo de manera implícita usando la fecha de devolución.

Persistencia y gestión de datos

La persistencia se implementó utilizando archivos planos en formato CSV, cumpliendo con el requisito del proyecto. Se definió un archivo independiente para cada almacenados en una carpeta data para centralizar la información.

Se utilizó el “;” como separador y las fechas se almacenan en formato ISO (yyyy-MM-dd), lo que facilita su manejo.

El DAO se encarga de realizar las operaciones CRUD, delegando la lectura y escritura de archivos a las clases de persistencia. Esta separación garantiza que cada clase tenga una única responsabilidad.

Los DAO trabajan con los DTO, ya que estos representan estructuras ideales de transferencia de datos. El Mapper centraliza la conversión entre modelos y DTO, reduciendo la duplicación

de código y facilitando futuras modificaciones en el formato de persistencia sin afectar la lógica del sistema. Finalmente, todo esto converge en la clase Biblioteca y la clase se usa para unir todo el modelo con el controlador.

Manejo de errores y excepciones

Con el fin de controlar situaciones excepcionales y evitar el rompimiento del código, se implementó un manejo de excepciones personalizadas las cuales detectan errores como entradas inválidas, registros inexistentes o violaciones a las reglas de negocio.

El uso de excepciones propias permite que el controlador muestre mensajes claros y comprensibles al usuario, manteniendo el flujo de ejecución controlado.

Gestión y uso de los archivos planos en trabajo colaborativo

Para el manejo de los archivos planos durante el desarrollo del proyecto, se utilizó una carpeta compartida a través de OneDrive. Esta decisión permitió que todos los integrantes del grupo tuvieran acceso en tiempo real a los archivos utilizados para la persistencia de la información.

Cada integrante del grupo contaba con permisos sobre la carpeta compartida, lo que facilitó el trabajo colaborativo y evitó problemas relacionados con versiones distintas de los archivos o pérdida de información. De esta forma, los cambios realizados por un integrante podían ser visualizados y utilizados por los demás de manera inmediata.

En el código, cada integrante configuró la ruta correspondiente a la carpeta compartida de OneDrive en su entorno local. Esto permitió que el sistema accediera a los mismos archivos planos, manteniendo la coherencia de los datos durante el desarrollo y las pruebas del programa.